Non Sequitur Publishing · White Paper

The State Coherence Problem: Write-Ahead-Log Replication and CRDT Reconciliation Under DDIL Connectivity

Justin H. Kuiper, CISSP

Written: 2026-05-11 · Published: 2026-05-13 · v1.0-preprint

Abstract

P3 established that custody-based transport — DTN custody transfer across heterogeneous, intermittent links — is the transport primitive for the tactical edge. Bundles arrive. State is moved. But moving state between disconnected nodes does not guarantee that the state held across those nodes constitutes a coherent operational picture. Two ships that have been operating independently for seventy-two hours, each accumulating sensor fusion data, targeting updates, and planning revisions from their own local sources, will not automatically produce a coherent combined operational picture when they reconnect — even if their custody transfers have been technically correct. They will produce a collision of locally-valid states, each internally consistent, together inconsistent, with no agreed mechanism for resolution.

This paper argues that state coherence at the tactical edge is a substrate-level problem, not an application-level concern, and that the two primitives required to address it — the Write-Ahead Log (WAL) for durable, ordered state recording and Conflict-Free Replicated Data Types (CRDTs) for deterministic, convergent state reconciliation — must be architectural first-class components of the tactical substrate, not optional add-ons that individual applications may or may not implement. Without these primitives, the operational picture assembled from state delivered by the P3 transport architecture is a probabilistic approximation of operational truth. With them, it is a deterministically convergent, governance-auditable representation of operational reality.

The paper develops the WAL as the primary substrate mechanism for state integrity within a single node: every write is logged before it takes effect, the log is append-only and cryptographically verifiable, and the current node state is fully reconstructible from the log at any point in time. The CRDT layer is then developed as the multi-node convergence mechanism: a class of data structures whose merge operations are commutative, associative, and idempotent, such that merging any two CRDT instances from any two nodes produces the same result regardless of merge order, with no central authority required and no window of connectivity required beyond eventual delivery. Together, WAL and CRDT constitute the state coherence substrate on which the AI Supervisor's operational reasoning (P5), the protocol selection mechanism (P6), and the workload class governance (P7) all depend.

How to cite

Justin H. Kuiper, CISSP (2026). *The State Coherence Problem: Write-Ahead-Log Replication and CRDT Reconciliation Under DDIL Connectivity* (v1.0-preprint). Non Sequitur Publishing. https://nonsequitur.tech/white-papers/state-coherence-problem/

ABSTRACT

Persistent state under partition is the most-studied problem in distributed systems and one of the least-applied to agentic operation. This paper argues that two well-understood primitives — write-ahead logs (WAL) and conflict-free replicated data types (CRDTs) — jointly solve the state coherence problem for agentic systems operating on the tactical substrate.[1][2] The argument is that WAL provides the durability and replay semantics required for governance event log integrity under partition; CRDTs provide the merge semantics required for distributed-state reconciliation upon reconnection; and the combination produces a persistence substrate that preserves HGC³AE²'s **C²** (Continuity) and **C³** (Coherence) commitments under DDIL conditions.

HOW TO CITE

Justin H. Kuiper, CISSP (2026). *The State Coherence Problem* (v1.0-preprint-draft). Non Sequitur Publishing.

1. INTRODUCTION

The cloud-native persistence model presumes a single authoritative tier with consistency semantics enforced by the underlying database.[3] When this tier is unreachable — as it is structurally on the tactical substrate (EDO-P1) — the agentic system must persist state locally and reconcile when reachability returns. The problem is not unfamiliar; distributed-systems literature has developed primitives for it across decades of refereed work.[4][5] The architectural commitment is to apply those primitives to agentic state explicitly.

§2 names five failure modes of cloud-native persistence under partition. §3 specifies WAL + CRDT as the architectural response. §4 treats coherence as continuous. §5 binds to Skipjack. §6 returns to HGC³AE².

2. FAILURE MODES OF CLOUD-NATIVE PERSISTENCE UNDER PARTITION

Lost writes. Writes that occur during partition and cannot be transmitted to the authoritative tier are lost when the local substrate restarts or fails over.

---

1 Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski, “Conflict-Free Replicated Data Types,” *SSS ’11*, 2011, https://doi.org/10.1007/978-3-642-24550-3_29.

2 C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz, “ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging,” *ACM Transactions on Database Systems* 17, no. 1 (1992): 94–162, https://doi.org/10.1145/128765.128770.

3 Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol, CA: O’Reilly, 2017).

4 Douglas B. Terry et al., “Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System,” *SOSP ’95*, 1995, https://doi.org/10.1145/224056.224070.

5 Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski, “Conflict-Free Replicated Data Types,” *SSS ’11*, 2011, https://doi.org/10.1007/978-3-642-24550-3_29.

Last-writer-wins divergence. Concurrent writes during partition merge with last-writer-wins semantics; the loser is silently discarded. For governance state, this is unacceptable.

Schema drift. Local state evolves during partition; reconciliation discovers schema differences that the authoritative tier cannot ingest.[6]

Replay incoherence. Without WAL discipline, the state at any moment is not deterministically reproducible from a known starting point plus a sequence of operations; recovery is ambiguous.

CAP-driven halt. A strict-consistency implementation halts under partition rather than admit divergence; mission-critical workloads do not tolerate halt.[7]

3. WAL + CRDT AS ARCHITECTURAL RESPONSE

3.1 WRITE-AHEAD LOGS

WAL provides **durability with replay semantics**: every state-modifying operation is logged before it is applied, the log is durable, and recovery from any failure point is achieved by replaying the log from the last consistent checkpoint.[8][9] For agentic state on the tactical substrate, WAL ensures that operations performed during partition are durably recorded; recovery from substrate failure replays the log to the same state.

WAL discipline includes operation ordering, checkpoint cadence, log compaction, and replication semantics. Each is well-developed in the peer-reviewed database literature; the architectural commitment is to apply WAL to agentic state as a first-class concern.

3.2 CONFLICT-FREE REPLICATED DATA TYPES

CRDTs provide **deterministic merge semantics** for concurrent updates. The class of data types includes counters, sets, registers, maps, and sequences; each is designed such that concurrent updates from multiple replicas converge to the same state regardless of merge order.[10][11]

---

6 Carlo A. Curino et al., “Schema Evolution for Databases and Applications: A Survey,” *Proceedings of the VLDB Endowment* 1, no. 1 (2008): 833–846, https://doi.org/10.14778/1453856.1453952.

7 Eric A. Brewer, “CAP Twelve Years Later: How the ‘Rules’ Have Changed,” *IEEE Computer* 45, no. 2 (2012): 23–29, https://doi.org/10.1109/MC.2012.37.

8 PostgreSQL Documentation, *Write-Ahead Logging (WAL)*, supplemented by Joseph M. Hellerstein, Michael Stonebraker, and James Hamilton, “Architecture of a Database System,” *Foundations and Trends in Databases* 1, no. 2 (2007): 141–259, https://doi.org/10.1561/1900000002.

9 Diego Ongaro and John Ousterhout, “In Search of an Understandable Consensus Algorithm (Extended Version),” *USENIX ATC ’14*, 2014.

10 Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski, “Conflict-Free Replicated Data Types,” *SSS ’11*, 2011, https://doi.org/10.1007/978-3-642-24550-3_29.

11 Martin Kleppmann and Alastair R. Beresford, “A Conflict-Free Replicated JSON Datatype,” *IEEE Transactions on Parallel and Distributed Systems* 28, no. 10 (2017): 2733–2746, https://doi.org/10.1109/TPDS.2017.2697382.

For agentic state under partition, CRDTs are the merge instruments for persistent memory, governance event logs (with append-only semantics), and intermediate work-in-flight state. CRDT discipline preserves coherence across reconciliation cycles without forcing strict consistency at write time.

3.3 THE COMBINED ARCHITECTURE

WAL + CRDT together produce: - **Durability** of every operation (WAL). - **Replay determinism** for recovery (WAL). - **Concurrent-update tolerance** during partition (CRDT). - **Deterministic merge** at reconciliation (CRDT). - **Coherence** across replicas without strict-consistency halt (combined).

The combination is not novel in distributed systems but is novel in being applied to agentic state explicitly as a substrate commitment.

4. COHERENCE AS CONTINUOUS PROCESS

Three phases recur. **Logging** at every state-modifying event; the WAL is the substrate's authoritative local record. **Merging** at reconnection; CRDTs reconcile concurrent updates deterministically. **Compaction** at periodic intervals; log compaction and CRDT tombstone collection prevent unbounded growth.

The continuous-process framing is the operational form of HGC^3AE^2's C^2 and C^3 under DDIL conditions: continuity through logged operations, coherence through deterministic merge.

5. SKIPJACK COUPLING

Skipjack mechanisms operate over the persistence substrate.[12] **Context integrity** is enforced by WAL replay determinism (the system can prove the state was reached from a known starting point through a known sequence of operations). **Structured routing** operates within the consistency envelope the persistence substrate provides. **HITL control points** fire on CRDT merge anomalies (concurrent updates that suggest envelope breach). **Iterative feedback** closes via the merged state informing envelope refinement.

6. IMPLICATIONS AND CONCLUSION

The implications extend to architecture (WAL and CRDT primitives are first-class), tooling (Postgres-style WAL, Automerge or Yjs-style CRDTs, etcd-style Raft for replication), and operational discipline (compaction, schema evolution, and merge audit are continuous concerns).

The core claim: **WAL + CRDT preserves HGC^3AE^2's C^2 and C^3 commitments under partition; the architectural commitment is to apply these well-understood primitives to agentic state explicitly**.

Returned to HGC^3AE^2 letter by letter:

H — Human-governed envelope updates flow through the WAL as durable governance events.

G — Curated Context is the merged state at any reconciliation point.

12 Justin H. Kuiper, CISSP, *Skipjack Protocol* (v1.0-preprint, 2026), zenodo.org/records/19869293.

C^1 — Cybersecurity of the persistence substrate is itself a load-bearing concern (MAO-P9, EDO-P8).

C^2 — Continuity is preserved by WAL durability under partition.

C^3 — Coherence is the paper's load-bearing commitment via CRDT merge discipline.

A — Agentically Engineered merge is deterministic, not heuristic.

E — Executed logging continuous at every state-modifying event.

The exponent — the closure — is the log-merge-adjust cycle.

The remaining EDO papers (orchestration, protocol selection, classification, accreditation, interoperability, deployment) operate on the persistence substrate this paper helps establish.[13][14]

REFERENCES

— end v1.0-preprint-draft —

Drafting state: v1.0-preprint draft. PR count: 12 inline (Shapiro SSS · Mohan TODS · Kleppmann · Terry SOSP · Brewer IEEE Computer · Curino VLDB · Hellerstein FnT · Ongaro USENIX ATC · Kleppmann/Beresford TPDS · Demers PODC · Fidge ACSC + supporting). §6 C^3 load-bearing.

13 EDO-P5 *Orchestration Problem* promotion (in concurrent drafting; yks-build#97).

14 EDO-P11 *Deployment Problem* CAPSTONE promotion (in concurrent drafting; yks-build#110).